

## LAB 3. LAYOUT, ROUTING & GIAO DIỆN BOOTSTRAP

Ở buổi học trước, chúng ta đã thành công hiển thị danh sách sản phẩm. Tuy nhiên, trang web trông còn khá đơn điệu và URL chưa được tối ưu. Buổi hôm nay sẽ tập trung vào việc tạo ra một giao diện nhất quán cho toàn bộ trang web, làm đẹp danh sách sản phẩm bằng Bootstrap và cấu hình URL thân thiện hơn với người dùng và công cụ tìm kiếm.

### MỤC TIÊU BÀI HỌC

Sau buổi học này, bạn sẽ có khả năng:

1. **Xây dựng một layout chung** (\_Layout.cshtml) để tái sử dụng Header, Footer, và Sidebar trên nhiều trang.
2. **Hiểu và cấu hình URL Routing** cơ bản trong ASP.NET Core MVC.
3. **Áp dụng các lớp (classes) của Bootstrap 5** để tạo giao diện dạng thẻ (Card) chuyên nghiệp cho sản phẩm.
4. **Sử dụng Tag Helpers** (asp-controller, asp-action) để tạo các liên kết (links) một cách an toàn và hiệu quả.
5. **Tích hợp hình ảnh sản phẩm** vào giao diện, làm cho trang web trở nên trực quan và hấp dẫn.

### YÊU CẦU KIẾN THỨC

- Đã hoàn thành bài thực hành số 2.
- Kiến thức cơ bản về HTML và CSS.
- Hiểu biết sơ lược về cách hoạt động của các trang web (ví dụ: các trang có chung phần đầu và phần cuối).

### CÔNG CỤ VÀ TÀI NGUYÊN

1. **Solution SportsStore** đã hoàn thành ở buổi 2.
2. **Bootstrap 5**: Thư viện CSS/JS phổ biến để xây dựng giao diện responsive.
3. **Hình ảnh sản phẩm mẫu**: Một vài file ảnh để hiển thị.

## NỘI DUNG

### Phần 1: Chuẩn bị tài nguyên tĩnh (Static Assets)

Trước khi code, chúng ta cần chuẩn bị "nguyên liệu" là Bootstrap và hình ảnh.

#### 1. Tải Bootstrap 5:

- Truy cập [trang chủ của Bootstrap](#).
- Tải về phiên bản đã được biên dịch (Compiled CSS and JS).
- Giải nén file zip. Bạn sẽ thấy 2 thư mục: css và js.

#### 2. Tổ chức thư mục wwwroot:

- Trong project SportsStore.WebUI, thư mục wwwroot là nơi chứa tất cả các file tĩnh (CSS, JS, ảnh, fonts...).
- Copy file bootstrap.min.css từ thư mục bạn vừa giải nén vào SportsStore.WebUI/wwwroot/css/.
- Copy file bootstrap.bundle.min.js vào SportsStore.WebUI/wwwroot/js/.

#### 3. Thêm hình ảnh sản phẩm:

- Tạo một thư mục con tên là images bên trong wwwroot.
- Tải về hoặc tự tạo một vài ảnh sản phẩm và đặt tên tương ứng với imageUrl trong FakeProductRepository (ví dụ: football.png, kayak.png...). Đặt chúng vào thư mục wwwroot/images/.

Cấu trúc thư mục wwwroot của bạn bây giờ sẽ trông giống như thế này:

```
wwwroot/  
├─ css/  
│   └─ bootstrap.min.css  
│       └─ site.css  
├─ js/  
│   └─ bootstrap.bundle.min.js  
│       └─ site.js  
├─ images/  
│   └─ football.png  
│   └─ surfboard.png  
│       └─ ...  
└─ lib/
```

## Phần 2: Hướng dẫn thực hành chi tiết

### Bước 1: Thiết kế Layout chung (\_Layout.cshtml)

Layout là một file Razor đặc biệt, hoạt động như một "template" cho các view khác. Các view con sẽ được "nhúng" vào vị trí @RenderBody().

1. Mở file \_Layout.cshtml trong thư mục Views/Shared.
2. Thay thế toàn bộ nội dung của file bằng đoạn code sau:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - SportsStore</title>

  <link rel="stylesheet" href="~/css/bootstrap.min.css" />
</head>
<body>
  <header class="bg-dark text-white p-2">
    <div class="container-fluid">
      <span class="navbar-brand ml-2">SPORTS STORE</span>
    </div>
  </header>

  <div class="container-fluid mt-4">
    <div class="row">
      <aside class="col-md-3">
        <p>Đây là Sidebar (sẽ được phát triển ở các buổi sau).</p>
        <p>Chúng ta có thể đặt các danh mục sản phẩm ở đây.</p>
      </aside>

      <main class="col-md-9">
        @RenderBody()
      </main>
    </div>
  </div>

  <footer class="border-top footer text-muted text-center p-3">
```

```
<div class="container">
    &copy; 2025 - SportsStore
</div>
</footer>

<script src="~/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

– Giải thích:

- ~/: Dấu ~ là một **Tag Helper** đặc biệt của ASP.NET Core, nó tự động trỏ đến thư mục gốc wwwroot. Điều này rất hữu ích vì bạn không cần lo lắng về đường dẫn tương đối.
- @ViewData["Title"]: Hiển thị tiêu đề được đặt trong từng View con.
- @RenderBody(): Đây là "trái tim" của layout. Toàn bộ nội dung của các View như Index.cshtml sẽ được chèn vào đúng vị trí này.

## Bước 2: Cấu hình Routing

ASP.NET Core MVC sử dụng một hệ thống routing mạnh mẽ để ánh xạ URL tới các action trong controller.

1. Mở file Program.cs trong project SportsStore.WebUI.
2. Tìm đến đoạn code cấu hình routing ở gần cuối file.

```
// ...
app.UseRouting();

// ...
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

– Giải thích Route mặc định:

- name: "default": Tên của route.
- pattern: "{controller=Home}/{action=Index}/{id?}": Đây là khuôn mẫu URL.

- Nó tìm kiếm một controller trong URL (ví dụ: /Product). Nếu không có, nó sẽ mặc định là Home.
- Tiếp theo, nó tìm kiếm một action (ví dụ: /List). Nếu không có, nó sẽ mặc định là Index.
- {id?}: id là một tham số tùy chọn (dấu ? có nghĩa là optional).
- Với route này, URL / sẽ được ánh xạ tới HomeController và action Index(), chính là trang chủ của chúng ta. Điều này đã đáp ứng được yêu cầu hiện tại. Chúng ta sẽ tùy chỉnh nó sâu hơn ở các buổi sau khi làm chức năng phân trang và lọc theo danh mục.

### Bước 3: Cải thiện View Index.cshtml với Bootstrap

Bây giờ, hãy làm cho danh sách sản phẩm trở nên đẹp mắt hơn bằng cách sử dụng component Card của Bootstrap.

1. Mở file Index.cshtml trong Views/Home.
2. Thay thế toàn bộ nội dung của file bằng code sau:

```
@model IQueryable<Product>

@{
    ViewData["Title"] = "Products";
}

<div class="row">
    @foreach (var p in Model)
    {
        <div class="col-md-4 col-sm-6 mb-4">
            <div class="card h-100 shadow-sm">
                

                <div class="card-body">
                    <h5 class="card-title">
                        @p.Name
                        <span class="badge bg-primary rounded-pill float-
end">

                            @p.Price.ToString("c")
                        </span>
                    </h5>
                </div>
            </div>
        </div>
    }
</div>
```

```
        </h5>
        <p class="card-text">@p.Description</p>
    </div>
    <div class="card-footer text-center">
        <a asp-controller="Cart" asp-action="AddToCart" asp-
route-productId="@p.ProductID"
            class="btn btn-success">
            Add To Cart
        </a>
    </div>
</div>
</div>
    }
</div>
```

#### Bước 4: Sử dụng Tag Helpers

Trong đoạn code trên, bạn đã thấy một ví dụ về Tag Helper:

```
<a asp-controller="Cart" asp-action="AddToCart" asp-route-
productId="@p.ProductID"
    class="btn btn-success">
    Add To Cart
</a>
```

– Giải thích Tag Helpers:

- `asp-controller="Cart"`: Chỉ định rằng liên kết này sẽ trỏ đến `CartController`.
- `asp-action="AddToCart"`: Chỉ định action `AddToCart` trong controller đó.
- `asp-route-productId="@p.ProductID"`: Thêm một tham số vào URL. ASP.NET Core sẽ tự động tạo ra một URL như `/Cart/AddToCart?productId=3`.
- **Lợi ích:** Thay vì phải viết URL thủ công (`href="/Cart/AddToCart?productId=..."`), Tag Helper giúp bạn tạo URL một cách an toàn. Nếu bạn đổi tên controller hoặc action, Tag Helper sẽ báo lỗi ngay khi biên dịch, giúp tránh các liên kết hỏng.

Để Tag Helpers hoạt động, hãy chắc chắn rằng file `_ViewImports.cshtml` trong thư mục Views có dòng sau:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
(Dòng này thường có sẵn khi bạn tạo project MVC mới).
```

## Bước 5: Chạy và kiểm tra kết quả

- Nhấn **F5** để chạy lại project SportsStore.WebUI.
- Bây giờ bạn sẽ thấy một trang web hoàn toàn mới:
  - Có Header màu tối và Footer ở cuối trang.
  - Có một Sidebar bên trái.
  - Các sản phẩm được hiển thị trong các thẻ Card đẹp mắt, có hình ảnh, mô tả, giá cả và nút "Add To Cart".
  - Giao diện sẽ tự động điều chỉnh (responsive) khi bạn thu nhỏ cửa sổ trình duyệt.

## Phần 3: Tổng kết & Mở rộng

### Kết quả

Bạn đã nâng cấp thành công ứng dụng của mình từ một trang danh sách đơn giản thành một trang web có cấu trúc chuyên nghiệp:

- **Giao diện nhất quán:** Nhờ `_Layout.cshtml`, tất cả các trang trong tương lai sẽ có cùng Header, Sidebar và Footer.
- **Trình bày chuyên nghiệp:** Bootstrap Card giúp hiển thị thông tin sản phẩm một cách rõ ràng và hấp dẫn.
- **URL thân thiện:** Hệ thống routing mặc định đã được thiết lập, sẵn sàng để mở rộng.
- **Mã nguồn dễ bảo trì:** Tag Helpers giúp việc tạo liên kết trở nên an toàn và dễ dàng hơn.

### Bài tập mở rộng

1. **Tùy chỉnh Sidebar:** Thay vì chỉ hiển thị văn bản tĩnh, hãy thử tạo một component (sẽ học ở buổi sau) hoặc đơn giản là hard-code một danh sách các danh mục (`<ul>`, `<li>`) trong `_Layout.cshtml` tại khu vực sidebar.
2. Thêm trang "About Us":
  - Tạo một action mới tên là `About()` trong `HomeController.cs`. Action này chỉ cần `return View();`.

- Tạo một View mới About.cshtml trong Views/Home. Thêm một vài nội dung giới thiệu về cửa hàng.
- Thêm một liên kết "About Us" vào Header trong \_Layout.cshtml sử dụng Tag Helper (<a asp-controller="Home" asp-action="About">About Us</a>).
- Chạy lại và kiểm tra xem bạn có thể điều hướng giữa trang chủ và trang "About Us" hay không, và cả hai trang đều có chung một layout.

### **TÀI LIỆU THAM KHẢO**

- [1]. Layout in ASP.NET Core: Microsoft Docs - Layout in ASP.NET Core
- [2]. Routing to controller actions in ASP.NET Core: Microsoft Docs - Routing
- [3]. Bootstrap 5 Card Component: Bootstrap Docs - Cards
- [4]. Tag Helpers in ASP.NET Core: Microsoft Docs - Tag Helpers